Jesús Ángel Bonilla Duran Viridiana Santibañez Carrillo Jocelyn Rodriguez Torres

UNIVERSIDAD POLITÉCNICA BICENTENARIO

Práctica 02. Llenado y vaciado automático

Presenta:

Jesús Ángel Bonilla Duran

Viridiana Santibáñez Carrillo

Jocelyn Rodriguez Torres

Matricula:

24033407

24033066

24033436

Asignatura:

Sistemas embebidos

Profesor:

Hugo Antonio Méndez Guzmán

Silao 15 de octubre de 2026

Jesús Ángel Bonilla Duran Viridiana Santibañez Carrillo Jocelyn Rodriguez Torres

Contenido

Introducción 3
Marco Teórico 3
Sistemas programables 3
Microcontrolador ESP32 3
Programación del ESP32 4
Simulación 4
Push buttons como entradas digitales 4
Sensor ultrasónico HC-SR04 5
Módulo relé y bombas de agua 5
Pantalla LCD con comunicación I2C 5
Desarrollo 6
Material requerido 6
Software utilizado 7
Descripción del código 11
Fórmulas utilizadas en el programa 11
Conexiones del circuito 12
Resultados 12
Observaciones y Conclusiones 14
Conclusiones 14
Referencias 15

Lista de Figuras

Figura 1 Simulación del circuito de control con sensor ultrasónico, pantalla LCD, botones pulsadores y módulos de relé mediante el ESP32 8
Figura 2 Bombas, mangueras y recipientes utilizados en el prototipo 12
Figura 3 Visualización de los parámetros del sistema en la pantalla LCD. 13
Figura 4 Montaje físico del sistema de llenado y vaciado automático. 13

Jesús Ángel Bonilla Duran Viridiana Santibañez Carrillo Jocelyn Rodriguez Torres

Introducción

El control automático de nivel de líquidos es una aplicación de la ingeniería que permite regular la cantidad de líquido contenida en un recipiente mediante sensores, controladores electrónicos y dispositivos de actuación este tipo de sistemas se utiliza en depósitos de agua, procesos industriales y diferentes aplicaciones donde es necesario mantener un nivel dentro de ciertos límites.

En esta práctica se desarrolló un prototipo de llenado y vaciado automático utilizando una tarjeta ESP32, un sensor ultrasónico HC-SR04, una pantalla LCD de 16 × 2 caracteres con módulo I2C, un módulo relé de dos canales y dos bombas de agua de corriente directa.

El sistema se programó para medir la distancia entre el sensor y la superficie del agua, calcular el porcentaje estimado de llenado y controlar las bombas de acuerdo con dos límites configurables se incorporaron tres pulsadores para modificar los límites inferior y superior durante la operación la pantalla LCD permite visualizar estos parámetros, así como el porcentaje calculado y el volumen aproximado del líquido contenido en el recipiente.

El objetivo principal fue implementar y evaluar un sistema capaz de controlar las etapas de llenado y vaciado de un recipiente mediante mediciones ultrasónicas y límites de operación establecidos en el programa

Marco Teórico

Sistemas programables

Un sistema programable es un sistema electrónico o computacional capaz de realizar diferentes funciones mediante un conjunto de instrucciones almacenadas en un programa a diferencia de un sistema de función fija, puede modificar su comportamiento al cambiar el programa sin necesidad de alterar físicamente sus componentes estos sistemas se utilizan en áreas como la robótica, automatización, control y dispositivos electrónicos.

Microcontrolador ESP32

El ESP32 es un microcontrolador y sistema en chip (SoC) desarrollado por Espressif Systems, que integra conectividad Wi-Fi y Bluetooth un SoC es un circuito integrado que reúne en una sola plataforma un sistema electrónico completo para una aplicación determinada, debido a sus características, el ESP32 puede utilizarse en diferentes tipos de

dispositivos y proyectos relacionados con sistemas embebidos e Internet de las Cosas (IoT) (Nick, s.f.).

Programación del ESP32

PlatformIO es un entorno de desarrollo utilizado para la programación de microcontroladores y diferentes plataformas de sistemas embebidos puede integrarse como una extensión en entornos de desarrollo como Visual Studio Code, proporcionando herramientas para crear y administrar proyectos, gestionar librerías, compilar programas y cargar el código en la tarjeta de desarrollo además, permite configurar cada proyecto mediante el archivo platformio.ini, donde se especifican aspectos como la placa y las características necesarias para su funcionamiento (PlatformIO, s.f.).

En este proyecto se utilizó PlatformIO dentro de Visual Studio Code para desarrollar y cargar los programas utilizados con el ESP32 su elección permitió mantener una estructura organizada del proyecto y facilitar la administración de las librerías y configuraciones necesarias para la programación también proporcionó herramientas como la compilación del código y el monitor serial, útiles para comprobar el funcionamiento del programa durante las pruebas (PlatformIO, s.f.).

Simulación

Wokwi es una plataforma en línea que permite diseñar, programar y simular proyectos electrónicos directamente desde un navegador web la herramienta permite trabajar con diferentes placas de desarrollo, entre ellas Arduino, ESP32, STM32 y Raspberry Pi Pico, además de integrar componentes electrónicos y conexiones para comprobar virtualmente el funcionamiento de un circuito (Group, 2023).

En este proyecto se utilizó Wokwi para diseñar y simular los circuitos empleados con el ESP32, permitiendo realizar pruebas del programa y verificar las conexiones de los componentes antes de llevar a cabo el montaje físico de esta manera, la simulación ayudó a detectar posibles errores de programación o conexión durante el desarrollo del proyecto.

Push buttons como entradas digitales

Un push button o botón pulsador es un interruptor eléctrico que funciona mediante una acción física de presión se utiliza en circuitos de control para iniciar o detener diferentes procesos al ejercer presión sobre el botón, su mecanismo interno modifica el estado de los contactos eléctricos, permitiendo cerrar o interrumpir el circuito (Group, 2023).

Los push buttons pueden encontrarse en configuraciones normalmente abiertas (NO) o normalmente cerradas (NC) en un contacto normalmente abierto, el circuito

permanece abierto mientras el botón no es presionado y los contactos se cierran al ejercer presión en cambio, un contacto normalmente cerrado permanece conectado mientras el botón está en reposo y se abre al presionarlo (Group, 2023).

Sensor ultrasónico HC-SR04

El HC-SR04 es un sensor que permite medir distancias mediante ondas ultrasónicas. Para realizar una medición, el microcontrolador envía un pulso al terminal TRIG, el sensor emite una onda sonora y, cuando recibe su reflexión, genera una señal en el terminal ECHO cuya duración está relacionada con el tiempo de recorrido de la onda.

En el prototipo, el sensor se coloca sobre el recipiente para medir la distancia hasta la superficie del agua cuando el nivel del líquido aumenta, esta distancia disminuye; cuando el nivel baja, la distancia aumenta.

La medición puede verse afectada por la posición del sensor, la inclinación de la superficie, las salpicaduras, los obstáculos y las características del recipiente.

Medición del nivel del tanque

La distancia obtenida por el sensor no representa directamente el porcentaje de llenado. Para convertirla en una variable más fácil de interpretar, se establecen dos referencias: una para el recipiente vacío y otra para el recipiente lleno.

En el programa se definieron los siguientes valores:

Distancia de referencia para el tanque lleno: 3.0 cm.

Distancia de referencia para el tanque vacío: 17.5 cm.

El programa limita el resultado para que permanezca entre 0 y 100 %. Si la distancia medida se encuentra fuera del intervalo de calibración, el porcentaje se ajusta al límite correspondiente.

Módulo relé y bombas de agua

Un módulo relé permite controlar una carga eléctrica mediante una señal de mando en este prototipo, las señales digitales del ESP32 se utilizan para controlar el módulo relé, que conmuta los circuitos de alimentación de las bombas.

Las dos bombas de corriente directa funcionan como actuadores una se encarga de introducir agua en el recipiente, mientras que la otra permite extraerla durante el vaciado.

El programa utiliza GPIO 23 para la señal de control de llenado y GPIO 19 para la señal de control de vaciado.

Pantalla LCD con comunicación I2C

La pantalla LCD de 16 × 2 carácter permite mostrar información mediante dos filas de dieciséis caracteres cada una el módulo I2C reduce la cantidad de conexiones

necesarias al utilizar principalmente dos líneas de comunicación: SDA y SCL en el código, la pantalla se declara con la dirección I2C 0x27 y se configura con las dimensiones de 16 columnas por 2 filas la comunicación se realiza mediante la biblioteca Wire.h, mientras que LiquidCrystal_I2C.h facilita el control de la pantalla.

La información se distribuye de la siguiente manera:

- Primera fila: límite inferior LL y porcentaje actual de llenado.
- Segunda fila: límite superior HL y volumen estimado en litros.
- Control mediante límites inferior y superior

El sistema utiliza dos límites de operación: un límite inferior LL y un límite superior HL. Inicialmente, el programa establece LL en 10 % y HL en 90 %.

La lógica de funcionamiento es la siguiente:

- Cuando el porcentaje calculado es menor o igual que LL, se activa el estado de llenado y se desactiva el vaciado.
- Cuando el porcentaje calculado es mayor o igual que HL, se activa el estado de vaciado y se desactiva el llenado.
- Cuando el porcentaje permanece entre ambos límites, se conserva el estado de operación anterior.

Esta estrategia crea un intervalo entre el inicio del llenado y el inicio del vaciado.

Desarrollo

Material requerido

- 1 tarjeta ESP32.
- Sensor ultrasónico HC-SR04.
- Pantalla LCD de 16 × 2 con módulo I2C
- Protoboard.
- Módulo relé de dos canales
- Bombas de agua de corriente directa
- Cables de conexión.
- Computadora con Visual Studio Code y PlatformIO.

- Simulador Wokwi.
- Recipientes de plástico
- Fuente de alimentación

Software utilizado

Para programar el sistema se utilizaron las siguientes herramientas:

- Visual Studio Code Se empleó como entorno de trabajo para editar y organizar los archivos del proyecto
- PlatformIO Permitió configurar la tarjeta, gestionar las dependencias y compilar el programa desarrollado para el ESP32.
- Arduino Framework Proporcionó las funciones utilizadas para configurar los pines, leer los pulsadores, controlar las salidas, medir tiempos y ejecutar el programa.
- Biblioteca Wire.h Se utilizó para la comunicación I2C con la pantalla LCD.
- Biblioteca LiquidCrystal_I2C.h Permitió inicializar y controlar la pantalla, ubicar el cursor y mostrar los valores calculados.
- Wokwi. Puede utilizarse para representar y simular el circuito, siempre que la configuración y los componentes necesarios estén disponibles en el simulador la simulación sirve como apoyo para revisar la lógica, pero no sustituye las pruebas físicas de las bombas y del circuito hidráulico.

Posteriormente, se realizó la simulación del circuito en Wokwi para comprobar el funcionamiento del sistema de control mediante el ESP32, el circuito está integrado por un sensor ultrasónico HC-SR04, una pantalla LCD, tres botones pulsadores, dos módulos de relé y dos zumbadores.

Jesús Ángel Bonilla Duran **Viridiana Santibañez Carrillo** **Jocelyn Rodriguez Torres**

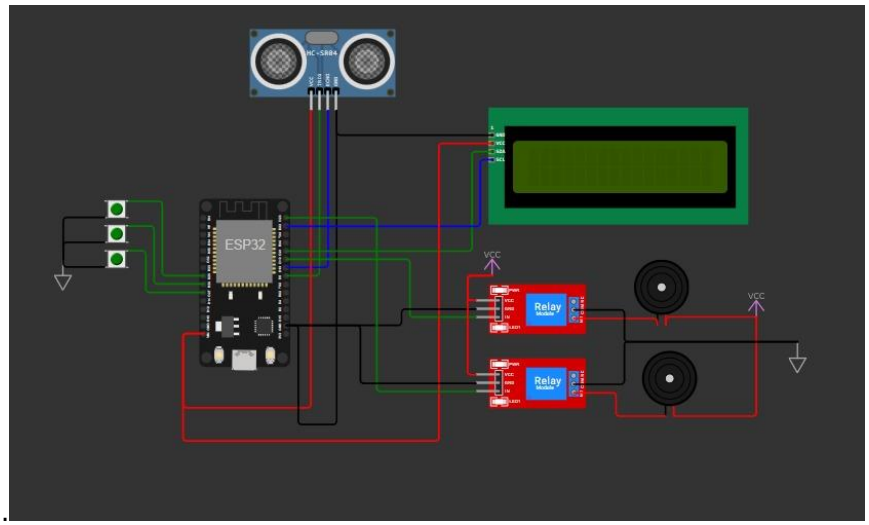


Figura 1 Simulación del circuito de control con sensor ultrasónico, pantalla LCD, botones pulsadores y módulos de relé mediante el ESP32

Mediante la programación establecida, se busca verificar la lectura del sensor, la visualización de información en la pantalla y la activación de las salidas de acuerdo con las condiciones programadas.

El archivo principal del programa es src/main.cpp el archivo platformio.ini contiene la configuración del entorno de compilación, la plataforma, la tarjeta seleccionada y las dependencias del proyecto.

A continuación, se muestra el código empleado para el desarrollo de la práctica.

main.cpp

```
#include <Arduino.h>
#include <Arduino.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

#define TRIG 5
#define ECHO 18

#define BOMBA_VACIADO 19
#define BOMBA_LLENADO 23
#define BOTON_SUBIR 25
#define BOTON_BAJAR 26
#define BOTON_SELECT 27
#define DISTANCIA_100 3.0
#define DISTANCIA_0 17.5
#define CAPACIDAD_LITROS 1.96

int LL = 10;
int HL = 90;

LiquidCrystal_I2C lcd(0x27, 16, 2);
bool llenando = false;
bool vaciando = false;
bool seleccionLL = true;
bool estadoAnteriorSubir = HIGH;
bool estadoAnteriorBajar = HIGH;
bool estadoAnteriorSelect = HIGH;
unsigned long tiempoParpadeo = 0;

bool mostrarSeleccion = true;
void setup() {
  pinMode(TRIG, OUTPUT);
  pinMode(ECHO, INPUT);
  pinMode(BOMBA_VACIADO, OUTPUT);
  pinMode(BOMBA_LLENADO, OUTPUT);
```

Jesús Ángel Bonilla Duran **Viridiana Santibañez Carrillo** **Jocelyn Rodriguez Torres**

```
  digitalWrite(BOMBA_VACIADO, LOW);
  digitalWrite(BOMBA_LLENADO, LOW);
  pinMode(BOTON_SUBIR, INPUT_PULLUP);
  pinMode(BOTON_BAJAR, INPUT_PULLUP);
  pinMode(BOTON_SELECT, INPUT_PULLUP);
  lcd.init();
  lcd.backlight();
  lcd.clear();

  delay(500);
}
void loop() {
  bool estadoSubir = digitalRead(BOTON_SUBIR);
  bool estadoBajar = digitalRead(BOTON_BAJAR);
  bool estadoSelect = digitalRead(BOTON_SELECT);
  if (estadoSelect == LOW && estadoAnteriorSelect == HIGH) {
    seleccionLL = !seleccionLL;
    mostrarSeleccion = true;
    tiempoParpadeo = millis();

    delay(150);
  }
  if (estadoSubir == LOW && estadoAnteriorSubir == HIGH) {
    if (seleccionLL) {
      if (LL < HL - 1) {
        LL++;
      }
    }
    else {

      if (HL < 100) {
        HL++;
      }
    }

    delay(150);
  }


  if (estadoBajar == LOW && estadoAnteriorBajar == HIGH) {

    if (seleccionLL) {
      if (LL > 0) {
        LL--;
      }
    }

    else {
      if (HL > LL + 1) {
        HL--;
      }
    }

    delay(150);
  }
  estadoAnteriorSubir = estadoSubir;
  estadoAnteriorBajar = estadoBajar;
  estadoAnteriorSelect = estadoSelect;

  if (millis() - tiempoParpadeo >= 500) {

    tiempoParpadeo = millis();

    mostrarSeleccion = !mostrarSeleccion;
  }

  digitalWrite(TRIG, LOW);

  delayMicroseconds(2);

  digitalWrite(TRIG, HIGH);

  delayMicroseconds(10);

  digitalWrite(TRIG, LOW);

  long tiempo = pulseIn(ECHO, HIGH);
  if (tiempo == 0) {
    return;
  }
  float distancia = tiempo * 0.0343 / 2;
  float porcentaje =
      (DISTANCIA_0 - distancia) *
      100.0 /
      (DISTANCIA_0 - DISTANCIA_100);
  if (porcentaje < 0) {
    porcentaje = 0;
  }

  if (porcentaje > 100) {
    porcentaje = 100;
```

Jesús Ángel Bonilla Duran **Viridiana Santibañez Carrillo** **Jocelyn Rodriguez Torres**

```
  }

  float litros =
      (porcentaje / 100.0) *
      CAPACIDAD_LITROS;

  if (porcentaje >= HL) {

    llenando = false;

    vaciando = true;
  }

  if (porcentaje <= LL) {

    vaciando = false;

    llenando = true;
  }

  if (llenando) {

    digitalWrite(BOMBA_LLENADO, HIGH);

    digitalWrite(BOMBA_VACIADO, LOW);
  }
  if (vaciando) {

    digitalWrite(BOMBA_LLENADO, LOW);

    digitalWrite(BOMBA_VACIADO, HIGH);
  }

  lcd.setCursor(0, 0);

  if (seleccionLL && !mostrarSeleccion) {

      lcd.print("LL=     ");

  } else {

    lcd.print("LL=");
    lcd.print(LL);
    lcd.print("%");
  }

  lcd.setCursor(8, 0);

  lcd.print("%=");

  lcd.print((int)porcentaje);

  lcd.print("%");
  lcd.setCursor(0, 1);

  if (!seleccionLL && !mostrarSeleccion) {

        lcd.print("HL=     ");

  } else {

    lcd.print("HL=");
    lcd.print(HL);
    lcd.print("%");
  }
  lcd.setCursor(8, 1);

  lcd.print("L=");

  lcd.print(litros, 2);

  lcd.print("  ");
  delay(200);
}
```

PRACTICA 2_SISTEMAS EMBEBIDOS

Jesús Ángel Bonilla Duran **Viridiana Santibañez Carrillo** **Jocelyn Rodriguez Torres**

Descripción del código

Tabla 1.funcionamientto de código

Instrucción o elemento	Función
`Arduino.h, Wire.h y LiquidCrystal_I2C.h`	Incorporan las funciones básicas del entorno Arduino y las herramientas para comunicarse con la pantalla LCD mediante I2C.
`Definición de pines con #define`	Asigna los GPIO del ESP32 al sensor ultrasónico, las bombas y los pulsadores.
`DISTANCIA_100, DISTANCIA_0 y CAPACIDAD_LITROS`	Establecen las distancias de referencia para el cálculo del nivel y la capacidad utilizada para estimar el volumen.
`Variables LL y HL`	Guardan los límites inferior y superior de operación, inicializados en 10 % y 90 %, respectivamente.
`Variables booleanas`	Almacenan los estados de llenado, vaciado, selección de límites y estados anteriores de los pulsadores.
`setup()`	Configura los pines, inicia la pantalla LCD y establece las salidas de las bombas en LOW al arrancar el programa.
`pinMode() y digitalRead()`	Configuran los pines y leen el estado lógico de los pulsadores.
`INPUT_PULLUP`	Activa la resistencia interna de polarización; al presionar un pulsador conectado a GND, la entrada se lee como LOW.
`Detección de pulsaciones`	Compara el estado actual y el anterior de cada botón para detectar una nueva pulsación y evitar incrementos continuos mientras se mantiene presionado.
`millis() y mostrarSeleccion`	Controlan el parpadeo del límite seleccionado en la pantalla LCD.
`digitalWrite(TRIG, ...) y pulseIn(ECHO, HIGH)`	Generan el pulso de activación del sensor y miden la duración de la señal de eco recibida.
`Cálculo de distancia`	Convierte el tiempo del eco en una distancia aproximada en centímetros.
`Cálculo del porcentaje`	Convierte la distancia medida en un porcentaje estimado de llenado usando las distancias de referencia.
`Condiciones de saturación`	Mantienen el porcentaje calculado dentro del intervalo de 0 a 100 %.
`Cálculo de litros`	Estima el volumen multiplicando la fracción de llenado por la capacidad configurada de 1.96 L.
`Condiciones porcentaje >= HL y porcentaje <= LL`	Cambian los estados de las bombas al alcanzar los límites superior e inferior.
`digitalWrite(BOMBA_LLENADO, ...) y digitalWrite(BOMBA_VACIADO, ...)`	Envían las señales de control al módulo relé para activar una bomba y desactivar la otra.
`lcd.init(), lcd.backlight() y lcd.clear()`	Inicializan la pantalla, encienden su iluminación y limpian su contenido al iniciar.
`lcd.setCursor() y lcd.print()`	Colocan y muestran en la pantalla los límites, el porcentaje y el volumen estimado.
`delay(150) y delay(200)`	Introducen pausas breves para el manejo de los pulsadores y el ciclo de ejecución del programa.

Fórmulas utilizadas en el programa

En el código se utiliza una aproximación de 0.0343 cm/µs para la velocidad del sonido. la distancia se obtiene en centímetros cuando el tiempo se expresa en microsegundos la fórmula que utiliza el programa es:

$$d = \frac{t * (0.0343)}{2}$$

- (d) es la distancia entre el sensor y el objeto.
- (t) es el tiempo de recorrido de la onda ultrasónica.
- (v) es la velocidad del sonido en el medio.

Para estimar el nivel del recipiente, el programa utiliza las distancias de referencia de 17.5 cm para el recipiente vacío y 3.0 cm para el recipiente lleno se utiliza:

$$Porcentaje = \frac{DISTANCIA_0 - distancia * 100.0}{DISTANCIA_0 - DISTANCIA_100}$$

Una vez que aplica la formula anterior ese porcentaje se convierte a litros usando:

$$Litros = \frac{Porcentaje}{100} * (Capacidad_LITROS)$$

La capacidad de litros = 1.96

Conexiones del circuito

Tabla 2 Configuración de pines del ESP32

Componente	Terminal o señal	GPIO del ESP32
Sensor HC-SR04	TRIG	5
Sensor HC-SR04	ECHO	18
Bomba de vaciado	Control del relé	19
Bomba de llenado	Control del relé	23
Pulsador SUBIR	Señal	25
Pulsador BAJAR	Señal	26
Pulsador SELECT	Señal	27
Pantalla LCD I2C	SDA	21
Pantalla LCD I2C	SCL	22

Los pulsadores se configuran con INPUT_PULLUP, por lo que deben conectarse de manera que al presionarlos la entrada correspondiente quede conectada a GND la pantalla LCD utiliza las líneas SDA y SCL para comunicarse con el ESP32.

Resultados

Durante las pruebas del prototipo se integraron el ESP32, el sensor ultrasónico, la pantalla LCD, los pulsadores y el módulo relé para realizar las funciones de medición, visualización y control.

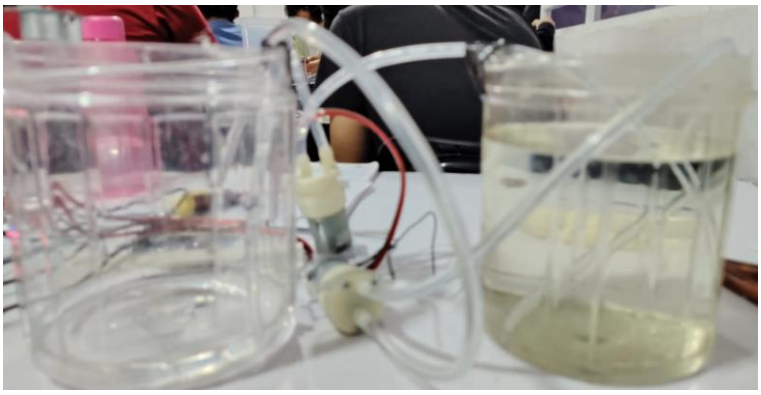


Figura 2 Bombas, mangueras y recipientes utilizados en el prototipo

El sensor ultrasónico se utilizó para obtener la distancia hasta la superficie del agua a partir de esa medición, el programa calculó el porcentaje estimado de llenado y el volumen aproximado del líquido estos valores se presentaron en la pantalla LCD, lo que permitió supervisar los parámetros del sistema.

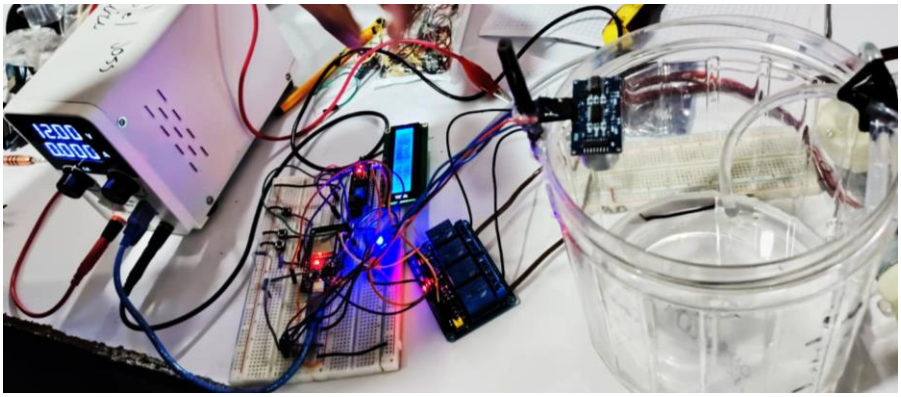


Figura 3 Visualización de los parámetros del sistema en la pantalla LCD.

La lógica programada establece que la bomba de llenado debe activarse cuando el porcentaje alcanza el límite inferior y que la bomba de vaciado debe activarse al llegar al límite superior entre ambos límites, el programa conserva el estado anterior de operación.

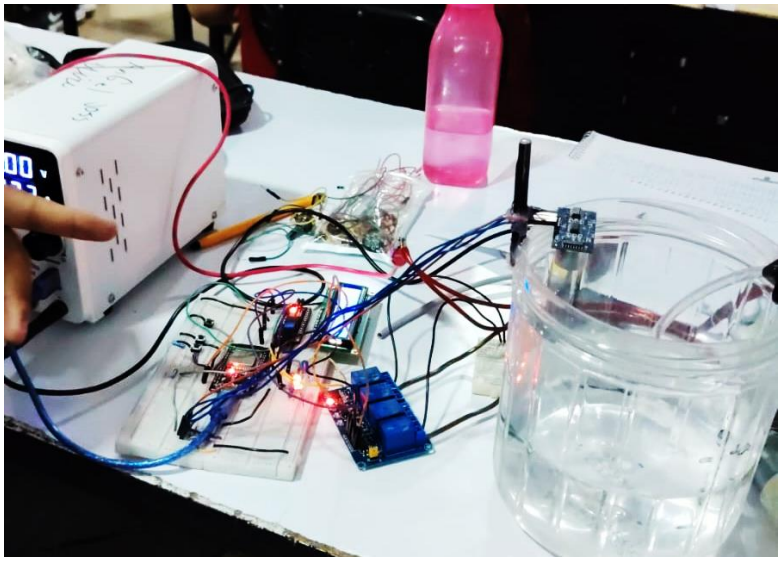


Figura 4 Montaje físico del sistema de llenado y vaciado automático.

En general, el prototipo integra los elementos necesarios para implementar el control automático de nivel la evaluación de su precisión depende de la calibración del sensor, de la capacidad real del recipiente y de la verificación del comportamiento de las bombas durante las pruebas.

Observaciones y Conclusiones

Durante el desarrollo de la práctica se identificó que la correcta conexión de los componentes y la configuración de los pines del ESP32 son fundamentales para el funcionamiento del sistema también fue necesario establecer distancias de referencia para convertir las mediciones ultrasónicas en un porcentaje estimado de llenado.

La pantalla LCD permitió consultar los parámetros sin depender exclusivamente de la computadora por su parte, los pulsadores facilitaron la modificación de los límites inferior y superior durante la operación.

La lógica de control permite conservar el estado de las bombas entre ambos límites, lo que evita cambios continuos de operación ante pequeñas variaciones del nivel, al iniciar el programa las dos variables de estado se encuentran desactivadas si la primera medición se ubica entre LL y HL, ninguna bomba se activa hasta alcanzar uno de los límites.

También se identificó una condición importante en el manejo de errores: cuando `pulseIn()` devuelve cero, el programa termina esa iteración sin actualizar las salidas ni la pantalla. Para una aplicación más confiable, sería recomendable incorporar un mecanismo de detección de lecturas inválidas y establecer una respuesta segura.

Conclusiones

Mediante esta práctica se desarrolló un prototipo de llenado y vaciado automático controlado por un ESP32 la integración del sensor ultrasónico HC-SR04 permitió medir la distancia hasta la superficie del agua y utilizar esta información para estimar el nivel del líquido.

El programa incorporó dos límites de operación, LL y HL, que determinan cuándo debe activarse la bomba de llenado y cuándo debe funcionar la bomba de vaciado, los pulsadores permiten modificar estos parámetros y la pantalla LCD facilita la consulta del porcentaje y del volumen estimado.

El desarrollo permitió aplicar conocimientos de programación de microcontroladores, electrónica digital, instrumentación y control de actuadores también

permitió comprender la relación entre la adquisición de una variable física, el procesamiento de los datos y la ejecución de una acción automática.

Como resultado, se obtuvo una estructura funcional de control que puede utilizarse como base para mejorar sistemas de regulación de nivel para incrementar su confiabilidad, es necesario considerar la calibración del sensor, la protección eléctrica de las entradas, el manejo de errores y la verificación de las condiciones de operación de las bombas.

Referencias

[1] (Nick, s.f.) *The ESP32 chip explained: Advantages and applications*.

[2] PlatformIO. (s.f.). PlatformIO: Professional collaborative platform for embedded development.

[3] Maxi. (s.f.). Introducción a PlatformIO con Visual Studio Code.

[4] Group, u. (2023). Wokwi: Simulador en línea de Arduino, ESP32, STM32 y Raspberry Pi Pico.